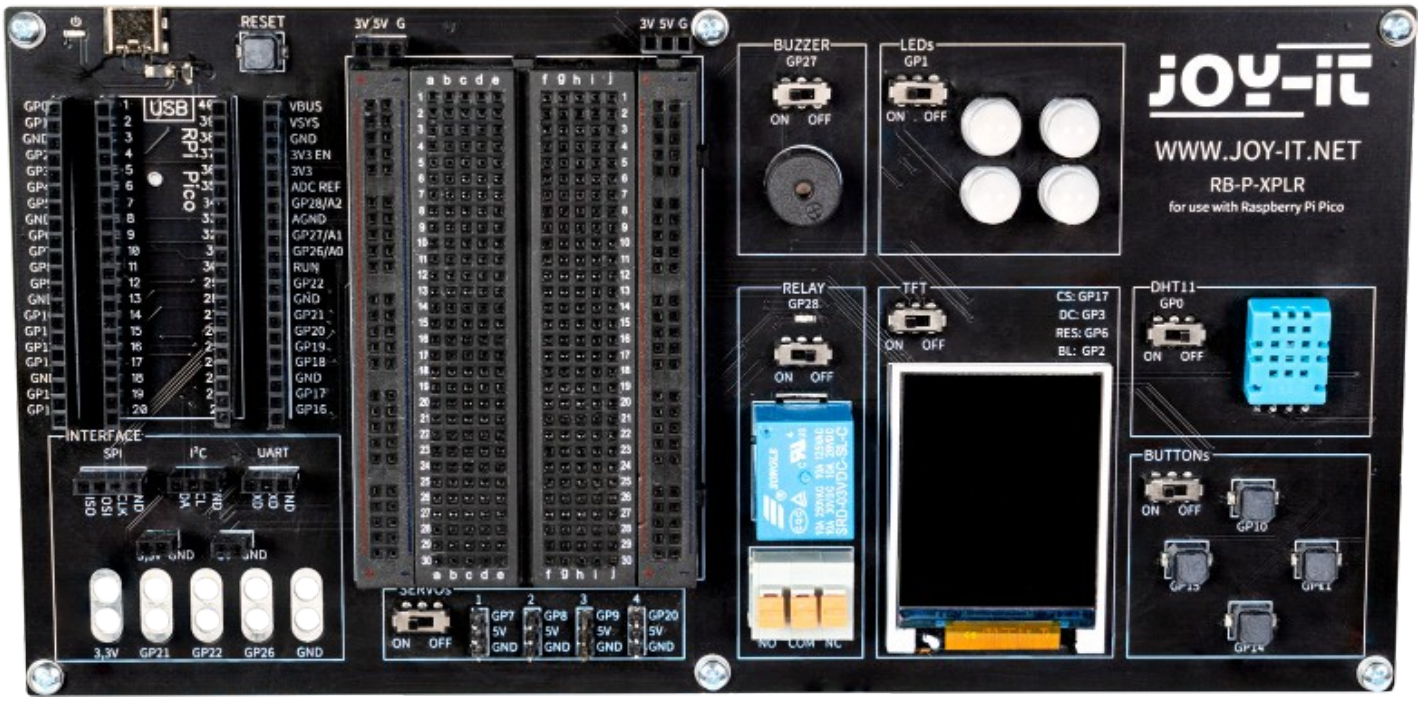
Livret de TP
Mallette Explorer Pico W

Bac Pro CIEL · Découvrir l'électronique programmée en MicroPython

Nom : ..... **Prénom :** .....

Classe : ..... **N° de mallette :** .....



Sommaire

TP 0	Découverte de la platine	p. 2
Aide	Mémo MicroPython et « Je suis bloqué »	p. 5
TP 1	Sorties : LED RGB et buzzer	p. 6
TP 2	Entrées : les boutons	p. 8
TP 3	Capteur, écran et alarme	p. 10
Bilan	Auto-évaluation	p. 12

Comment utiliser ce livret

- Lisez **toute** la consigne avant de taper du code.
- Répondez aux questions sur les lignes prévues.
- Les encadrés **Coup de pouce** vous aident quand vous bloquez.
- Un programme ne marche pas ? Allez à la **page 5**.
- ✔ = faites valider par le professeur avant de continuer.

Règles de sécurité : à respecter à chaque séance

- L'USB-C de la platine sert **seulement à alimenter**. Pour programmer, branchez le **micro-USB du Pico** au PC.
- On insère ou retire le Pico **hors tension**.
- **3,3 V maximum** sur une broche GPIO, jamais 5 V.
- Le relais ne commande **jamais le 230 V** du secteur.
- En fin de séance : tout est rangé dans la mallette.

TP 0 Découverte de la platine

Objectif du TP 0 : connaître le matériel, installer les outils et faire fonctionner un premier programme. **Durée :** 2 h.

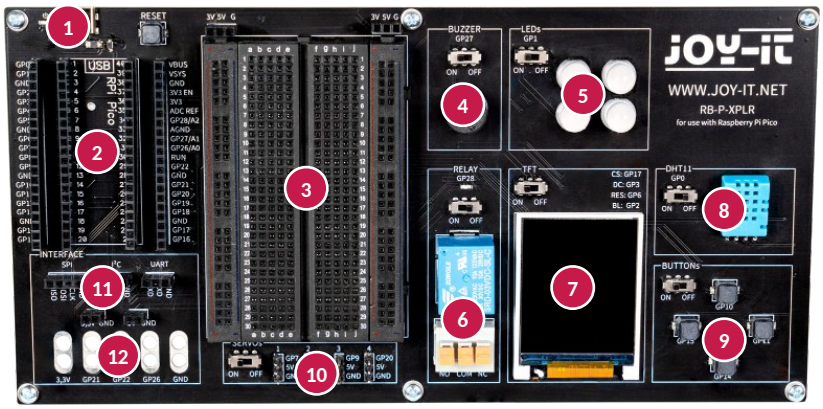
Exercice 1 · Inventaire de la mallette

Cochez chaque élément présent. S'il en manque un, prévenez le professeur **avant** de commencer.

- ☐ Raspberry Pi Pico W
- ☐ Cordon micro-USB
- ☐ Capteur UV S12SD
- ☐ Cordons crocodile et Dupont
- ☐ Platine Explorer
- ☐ Servomoteur + roue
- ☐ Encodeur rotatif
- ☐ Alimentation USB-C
- ☐ Capteur ultrason SEN-US01
- ☐ Sachet de composants

Exercice 2 · Repérer les modules de la platine

Chaque numéro sur la photo désigne un élément. Complétez le tableau avec les noms de la liste, puis relevez la broche (GPIO) écrite sur la platine et indiquez s'il s'agit d'une **entrée** ou d'une **sortie**.



Liste des noms :

Alimentation USB-C · Support du Pico · Breadboard · Buzzer · 4 LED RGB · Relais · Écran TFT · Capteur DHT11 · 4 boutons · 4 servomoteurs · Connecteurs SPI/I²C/UART · Pinces crocodile

N°	Nom de l'élément	Broche(s) GPIO	Entrée / Sortie
1	*Alimentation USB-C*	—	—
2			
3			
4	*Buzzer*	*GP27*	*Sortie*
5			
6			
7			
8			
9			
10			
11			
12			

Coup de pouce

Les noms sont sérigraphiés en anglais sur la platine : BUZZER, RELAY (relais), BUTTONS (boutons), SERVOS… Les lignes 1 et 4 sont remplies pour vous montrer l'exemple.

À savoir

Le **Raspberry Pi Pico W** est un **microcontrôleur** : un petit ordinateur dans une seule puce, qui exécute un seul programme en boucle. On le programme en **MicroPython** avec le logiciel **Thonny**. Un fichier nommé **`main.py`** enregistré sur le Pico démarre **tout seul** à chaque mise sous tension.

Exercice 3 · Installer MicroPython sur le Pico

1. Vérifiez que le Pico est bien enfoncé sur la platine (câbles **débranchés**).
2. **Maintenez** le bouton blanc **BOOTSEL** du Pico, branchez le câble **micro-USB** au PC, puis relâchez.
3. Dans Thonny : **Outils › Options › Interpréteur**, choisissez **MicroPython (Raspberry Pi Pico)**.
4. Cliquez sur **Installer ou mettre à jour MicroPython**, variante **Raspberry Pi · Pico W / Pico WH**, puis **Installer**.
5. Fermez la fenêtre : en bas de Thonny, la console affiche **>>>**.

Exercice 4 · Premier contact avec la console

Tapez chaque commande dans la console (après **>>>**), appuyez sur Entrée et notez le résultat.

`print("Bonjour CIEL")`	`6 * 7`	`2 ** 10`
→	→	→ .....

Exercice 5 · Faire clignoter la LED du Pico

Dans la zone d'édition de Thonny, tapez ce programme. Enregistrez-le **sur le Raspberry Pi Pico** sous le nom **`main.py`**, puis cliquez sur **Exécuter** (bouton vert ▶ ou touche F5).

```
from machine import Pin
from time import sleep

led = Pin("LED", Pin.OUT)

while True:
    led.toggle()
    sleep(0.5)
```

Lexique

`Pin(…, Pin.OUT)` : déclare une sortie
`while True:` : répète à l'infini
`sleep(0.5)` : attend 0,5 s
Les **4 espaces** devant une ligne indiquent qu'elle est dans la boucle.

Q1. Que fait l'instruction **`led.toggle()`** ? Observez la LED pour répondre.

.....

Q2. Modifiez le programme pour que la LED clignote **deux fois plus vite**. Quelle ligne avez-vous changée ?

.....

Q3. Débranchez puis rebranchez le Pico. Le programme redémarre-t-il seul ? Pourquoi ?

.....

Coup de pouce

Thonny affiche une erreur ou rien ne se passe ? Vérifiez en bas à droite de Thonny qu'il est écrit **MicroPython (Raspberry Pi Pico)**, puis cliquez sur **Stop**. Les erreurs courantes sont expliquées **page 5**.

✔ **Point de validation :** la LED du Pico clignote deux fois plus vite. Visa professeur ☐

Exercice 6 · Installer le programme de démonstration

1. Récupérez le dossier **Demo_JoyIT** donné par le professeur. Il contient **`main.py`**, le dossier **`lib`** et le fichier **`font5x7.fnt`**.
2. Dans Thonny, ouvrez **Affichage › Fichiers**. En haut : votre PC. En bas : le Pico.
3. Faites un clic droit sur chacun des 3 éléments puis **Téléverser vers /** pour les copier sur le Pico.
4. Ouvrez **`main.py` depuis le Pico** et exécutez-le.

Questions · Le programme plante : on enquête !

Q4. Recopiez la **dernière ligne** du message d'erreur affiché en rouge dans la console.

Q5. À quelle **ligne** du programme l'erreur se produit-elle ? .....

Q6. Expliquez avec vos mots pourquoi le programme plante à cet endroit.

Q7. Corrigez le programme pour qu'il fonctionne. Décrivez votre correction.

Q8. Une fois corrigé, que fait ce programme de démonstration ? Citez 4 modules qui s'activent.

Coup de pouce

Le message d'erreur commence par son **type** (**`NameError`** = nom inconnu) puis donne le **nom** en cause. Utilisez **Ctrl + F** dans Thonny pour chercher la ligne où cette variable reçoit une valeur pour la première fois. Est-ce **avant** ou **après** la ligne qui plante ?

Exercice 7 · À quoi servent les interrupteurs ?

Programme de démo en marche, basculez un par un ces interrupteurs sur **OFF**, observez, puis remettez-les sur **ON**.

Interrupteur sur OFF	Ce que j'observe
BUZZER	
LEDs	
RELAY (relais)	
TFT (écran)	

Q9. À votre avis, pourquoi peut-on désactiver chaque module ?

✔️ **Point de validation :** la démo fonctionne et le tableau des interrupteurs est rempli. Visa professeur ☐

AIDE

Mémo MicroPython et « Je suis bloqué »

À garder sous la main

Les instructions de base

```
from machine import Pin, PWM
from time import sleep

# Sortie : 1 = actif, 0 = inactif
relais = Pin(28, Pin.OUT)
relais.value(1)

# Entree avec pull-down
bp = Pin(10, Pin.IN, Pin.PULL_DOWN)
etat = bp.value()   # 0 ou 1

# Condition
if etat == 1:
    print("appui")
else:
    print("repos")

# Repeter 4 fois (i = 0,1,2,3)
for i in range(4):
    print(i)
```

Les modules de la platine

```
# 4 LED RGB (GP1)
from neopixel import NeoPixel
leds = NeoPixel(Pin(1), 4)
leds[0] = (60, 0, 0)  # R, G, B
leds.write()          # envoi !

# Buzzer (GP27)
bz = PWM(Pin(27))
bz.freq(440)          # en Hz
bz.duty_u16(32768)    # son
bz.duty_u16(0)        # silence

# Capteur DHT11 (GP0)
from dht import DHT11
capteur = DHT11(Pin(0))
capteur.measure()
t = capteur.temperature()
h = capteur.humidity()

# Boutons : GP10 GP11 GP14 GP15
```

Je suis bloqué : je lis le message d'erreur

Ce que je vois	Cause probable	Ce que je fais
`Pas de >>> ou « Device not connected »`	Le Pico n'est pas reconnu	Câble micro-USB (pas l'USB-C !), bouton Stop, vérifier l'interpréteur
`SyntaxError`	Faute d'écriture	Oubli de « : », d'une parenthèse ou d'un guillemet
`IndentationError`	Décalage incorrect	4 espaces sous chaque ligne qui finit par « : »
`NameError: name 'x' isn't defined`	Variable inconnue	Faute de frappe, majuscule, ou variable utilisée avant d'être créée
`ImportError: no module named …`	Bibliothèque absente	Copier le dossier lib sur le Pico
`OSError (capteur DHT11)`	Mesure ratée	Interrupteur DHT11 sur ON, 2 s entre deux mesures
`Aucune erreur, mais rien ne marche`	Module désactivé ou envoi oublié	Interrupteur du module sur ON ? leds.write() oublié ?
`Le programme ne s'arrête pas`	Boucle infinie	Bouton Stop ou Ctrl + C

Toujours bloqué ? Relisez la consigne, comparez votre code ligne par ligne avec l'exemple, demandez à votre voisin, puis au professeur.

TP 1 Sorties : les LED RGB

Objectif du TP 1 : commander des sorties (LED et buzzer) pour réaliser un feu tricolore sonore pour piétons malvoyants.
Durée : 2 h. **Interrupteurs :** LEDs et BUZZER sur ON.

À savoir : la synthèse additive

Chaque LED RGB contient 3 petites LED : **R**ouge, **V**erte (G en anglais) et **B**leue. En réglant l'intensité de chacune de **0 à 255**, on obtient toutes les couleurs. Les 4 LED sont **chaînées** sur une seule broche (GP1) : on prépare les couleurs avec `leds[i] = (R, G, B)`, puis on les envoie toutes avec `leds.write()`.



Q1. Complétez le code couleur (valeurs de 0 à 60 pour ne pas être ébloui) et vérifiez en testant.

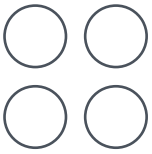
Rouge	Vert	Bleu	Jaune	Violet	Blanc	Éteint
(60, 0, 0)	(, ,)	(, ,)	(, ,)	(, ,)	(, ,)	(, ,)

Exercice 1 · Allumer une LED

```
from machine import Pin
from neopixel import NeoPixel

leds = NeoPixel(Pin(1), 4)
leds[0] = (60, 0, 0)
leds.write()
```

Q2. Quelle LED s'allume ? Testez `leds[1]`, `leds[2]` et `leds[3]`, puis écrivez le numéro de chaque LED dans son cercle :



Exercice 2 · Le chenillard

```
from time import sleep
while True:
    for i in range(4):
        leds[i] = (0, 0, 60)
        leds.write()
        sleep(0.3)
        leds[i] = ???
        leds.write()
```

Ajoutez ces lignes à la suite de l'exercice 1. Remplacez **???** pour qu'une seule LED bleue « avance » à la fois.

Q3. Qu'avez-vous écrit à la place de **???** ?

.....

Exercice 3 · Le feu tricolore

```
def toutes(couleur):
    for i in range(4):
        leds[i] = couleur
    leds.write()
while True:
    toutes((0, 60, 0))  # vert
    sleep(3)
    # a completer...
```

Une **fonction** est un bloc de code qu'on écrit une fois et qu'on réutilise : `toutes()` allume les 4 LED de la même couleur.

Complétez : **vert 3 s, orange 1 s, rouge 3 s**, en boucle.

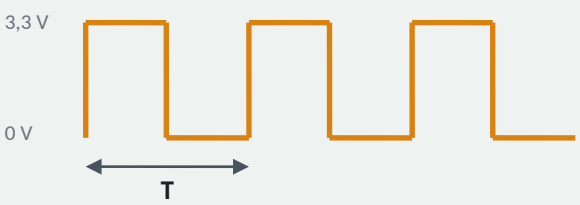
Coup de pouce : l'orange, c'est beaucoup de rouge et un peu de vert, essayez `(60, 20, 0)`.

✔ **Point de validation :** le feu tricolore fonctionne (vert, orange, rouge). Visa professeur ☐

À savoir : le signal PWM

Le buzzer reçoit un **signal carré** : la broche passe très vite de 0 V à 3,3 V.

- La **période T** est la durée d'un motif complet.
- La **fréquence** f = 1 / T, en hertz (Hz), fixe la **hauteur du son** (grave ou aigu).
- Le **rapport cyclique** est le pourcentage du temps passé à 3,3 V. Avec `duty_u16()` : 0 = 0 % et 65535 = 100 %.



Q1. Le La du diapason vaut 440 Hz. Sa période : T = 1 / f = ... ms

Q2. `duty_u16(32768)` donne un rapport cyclique de ..... %.

Exercice 4 · Premier bip et test des fréquences

```
from machine import Pin, PWM
from time import sleep

bz = PWM(Pin(27))
bz.freq(440)
bz.duty_u16(32768)
sleep(1)
bz.duty_u16(0)
```

Testez le programme puis changez la fréquence. Notez si le son est **grave**, **moyen** ou **aigu**.

Fréquence	Son entendu
200 Hz	
440 Hz	
1 000 Hz	
3 000 Hz	

Exercice 5 · Le feu pour piétons malvoyants

```
def bip(frequence, duree):
    bz.freq(frequence)
    bz.duty_u16(32768)
    sleep(duree)
    bz.duty_u16(0)
```

Ajoutez la fonction `bip()` à votre feu tricolore. Pendant le **rouge** (3 s), le buzzer doit émettre **6 bips courts** : un bip de 0,1 s toutes les 0,5 s.

Indice : pour répéter 6 fois, utilisez `for _ in range(6):`*. Combien doit durer la pause après chaque bip ?*

Défi · Jouer « Au clair de la lune »

Note	Do	Ré	Mi	Fa	Sol	La	Si
Fréquence (Hz)	262	294	330	349	392	440	494

Mélodie : **Do Do Do Ré Mi**(long) **Ré**(long) **Do Mi Ré Ré Do**(long). Astuce : rangez les notes dans une liste et parcourez-la avec une boucle `for`.

✔ **Point de validation :** le feu tricolore bipe pendant le rouge. Visa professeur ☐

TP 2 Entrées : lire un bouton

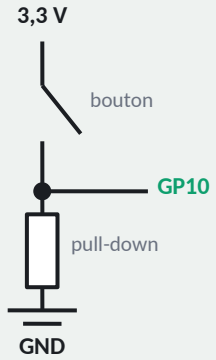
Objectif du TP 2 : lire des entrées (boutons) pour créer un mini-piano et un jeu de réflexes. **Durée :** 2 h. **Interrupteurs :** BUTTONs, LEDs et BUZZER sur ON.

À savoir : entrée numérique et résistance de tirage

Une **entrée numérique** ne lit que deux valeurs :

- **0** quand la broche est à 0 V ;
- **1** quand elle est à 3,3 V.

Sur la platine, chaque bouton relie la broche au **3,3 V**. Bouton relâché, la broche serait « en l'air » et lirait n'importe quoi : la **résistance pull-down** (activée par `Pin.PULL_DOWN`) la ramène alors à **0 V**.



Questions · Avant de programmer

Q1. Bouton **relâché** :
tension sur GP10 = …… V, valeur lue = ……

Q2. Bouton **appuyé** :
tension sur GP10 = …… V, valeur lue = ……

Exercice 1 · Afficher l'état d'un bouton

```
from machine import Pin
from time import sleep
bp = Pin(10, Pin.IN, Pin.PULL_DOWN)
while True:
    print(bp.value())
    sleep(0.2)
```

Lancez le programme et appuyez sur le bouton : les valeurs défilent dans la console.

Astuce : **Affichage › Traceur** dessine la courbe de la valeur.

Exercice 2 · Retrouver la broche de chaque bouton

Changez le numéro dans `Pin(10, …)` pour trouver quelle broche correspond à chaque bouton.

Bouton	Haut	Droite	Bas	Gauche
Broche GPIO	GP	GP	GP	GP

Exercice 3 · Un bouton qui allume les LED

```
while True:
    if bp.value() == 1:
        toutes((0, 60, 0))
    else:
        ???
    sleep(0.05)
```

Bouton **haut** appuyé : les 4 LED s'allument en vert. Relâché : elles s'éteignent. Reprenez la fonction `toutes()` du TP 1 et complétez `???`.

Coup de pouce : pensez aux imports et à la création de `leds` en haut du programme (mémo page 5).

Bonus : une couleur par bouton.

✔ **Point de validation :** le tableau des boutons est rempli et l'exercice 3 fonctionne. Visa professeur ☐

Exercice 4 · Le mini-piano

```
boutons = [Pin(n, Pin.IN, Pin.PULL_DOWN)
           for n in (10, 11, 14, 15)]
NOTES = [262, 294, 330, 349]  # Do Re Mi Fa

while True:
    appui = None
    for i in range(4):
        if boutons[i].value() == 1:
            appui = i
    if appui is None:
        bz.duty_u16(0)
    else:
        bz.freq(???)
        bz.duty_u16(32768)
    sleep(0.02)
```

Chaque bouton joue une note. Les broches et les notes sont rangées dans des **listes** : `NOTES[0]` vaut 262, `NOTES[1]` vaut 294…

Complétez `???`.

Q1. Que contient la variable `appui` quand on presse le bouton bas ?

Q2. Et quand aucun bouton n'est pressé ?

À savoir : mesurer une durée

`ticks_ms()` donne le nombre de millisecondes écoulées depuis le démarrage du Pico. `ticks_diff(fin, debut)` calcule l'écart entre deux instants. `random.randint(2000, 5000)` tire un nombre entier au hasard entre 2000 et 5000.

Exercice 5 · Le jeu de réflexes

Cahier des charges : les LED s'allument en vert après un délai **aléatoire** de 2 à 5 s. Le joueur appuie le plus vite possible sur un bouton. Le programme affiche le temps de réaction en ms.

```
from time import ticks_ms, ticks_diff
import random

while True:
    toutes((0, 0, 0))
    sleep(random.randint(2000, 5000) / 1000)
    toutes((0, 60, 0))
    debut = ticks_ms()
    while boutons[0].value() == 0:
        pass        # on attend l'appui
    temps = ???
    print("Reaction :", temps, "ms")
    sleep(1)
```

Q3. Notez vos 3 essais :

Essai 1 : …… ms

Essai 2 : …… ms

Essai 3 : …… ms

Améliorations :

- accepter n'importe quel bouton ;
- détecter le joueur qui appuie trop tôt ;
- garder le meilleur temps.

Défi · Utiliser une interruption

```
def appui(pin):
    print("Bouton !")

bp.irq(trigger=Pin.IRQ_RISING,
       handler=appui)
```

Avec une **interruption**, c'est le bouton qui prévient le programme : plus besoin de le surveiller en boucle. Testez puis expliquez pourquoi « Bouton ! » s'affiche parfois plusieurs fois (le **rebond**).

✔ **Point de validation :** le mini-piano fonctionne et le temps de réaction s'affiche. Visa professeur ☐

TP 3 Capteur : mesurer et afficher

Objectif du TP 3 : réaliser une station météo qui déclenche une alarme au-delà d'une température réglable. **Durée :** 3 h.
Interrupteurs : DHT11, TFT, RELAY, BUTTONs, LEDs, BUZZER sur ON.

À savoir : le capteur DHT11

Le DHT11 mesure la **température** et l'**humidité** de l'air. Il envoie ses mesures en numérique sur un seul fil (GP0). Il ne faut pas l'interroger plus d'**une fois par seconde**.

Grandeur	Plage	Précision
Température	0 à 50 °C	± 2 °C
Humidité	20 à 80 %	± 5 %

Exercice 1 · Lire le capteur

```
from machine import Pin
from time import sleep
from dht import DHT11

capteur = DHT11(Pin(0))

while True:
    try:
        capteur.measure()
        t = capteur.temperature()
        h = capteur.humidity()
        print("T =", t, "C  H =", h, "%")
    except OSError:
        print("Erreur de lecture")
    sleep(2)
```

`try` / `except` : si la mesure échoue, le programme affiche un message **au lieu de s'arrêter**.

Q1. Mesures de la salle :
T = ... °C H = ... %

Q2. Soufflez doucement sur le capteur. Que se passe-t-il ?

.....

Q3. Le capteur affiche 22 °C. La vraie température est comprise entre ..... °C et ..... °C.

Exercice 2 · Afficher sur l'écran

Copiez d'abord sur le Pico le dossier `lib` (avec `ST7735.py`) et le fichier `font5x7.fnt` (voir TP 0, exercice 6).

```
from machine import SPI
import ST7735

Pin(2, Pin.OUT).value(1)   # retroeclairage
spi = SPI(0, baudrate=8000000, sck=Pin(18),
          mosi=Pin(19), miso=Pin(16))
ecran = ST7735.ST7735(spi, rst=6, ce=17, dc=3)
ecran.reset()
ecran.begin()
ecran.p_string(10, 40, "Temp : " + str(t) + " C")
```

Ajoutez l'initialisation **avant** la boucle et l'affichage **dans** la boucle.

`p_string(x, y, texte)` écrit à **x** pixels du bord gauche et **y** pixels du haut. L'écran fait 128 × 160 pixels.

Ajoutez une ligne pour l'**humidité**, 20 pixels plus bas.

Coup de pouce

Écran blanc ? Interrupteur TFT sur ON et rétroéclairage `Pin(2)` à 1. `ImportError` ? Le dossier `lib` n'est pas sur le Pico.

Alarme : décider, agir et valider

Cahier des charges de la station météo

- Si la température **dépasse le seuil** : LED rouges, buzzer actif, **relais activé** (il pourrait commander un ventilateur).
- Sinon : LED vertes, buzzer silencieux, relais désactivé.
- Le seuil (26 °C au départ) se règle avec les boutons **haut** (+1) et **bas** (−1) et s'affiche sur l'écran.

Exercice 3 · Programmer l'alarme

Complétez l'algorithme, puis traduisez-le en MicroPython dans votre programme.

SI t > seuil **ALORS**

LED en …… , buzzer …… , relais ……

SINON

LED en …… , buzzer …… , relais ……

FIN SI

Pour le relais : `relais = Pin(28, Pin.OUT)` *puis* `relais.value(1)` *ou* `relais.value(0)`*. On entend un « clic » quand il commute.*

Exercice 4 · Régler le seuil avec les boutons

Ajoutez la lecture des boutons haut et bas dans la boucle pour modifier la variable `seuil`, puis affichez-la sur l'écran.

Problème : avec `sleep(2)` dans la boucle, les boutons réagissent mal. Proposez une solution :

.....

Exercice 5 · Valider le fonctionnement (protocole de test)

Réalisez chaque test et complétez le tableau. Un système est **conforme** quand le résultat observé est celui attendu.

Test	Action	Résultat attendu	Résultat observé	Conforme ?
1	Mettre en marche, T < seuil	*LED vertes, silence, relais au repos*		oui / non
2	Chauffer le capteur avec les doigts jusqu'à T > seuil			oui / non
3	Appuyer 2 fois sur le bouton haut			oui / non
4	Laisser refroidir sous le seuil			oui / non

Défi · L'hystérésis

Quand T oscille autour du seuil, le relais claque sans arrêt. Solution : on déclenche à **seuil + 1** et on arrête à **seuil − 1**. Modifiez votre programme pour obtenir ce comportement.

✔ **Point de validation :** la station météo est conforme aux 4 tests. Visa professeur ☐

BILAN

Bilan et auto-évaluation

Questions de synthèse

S1. Quelle est la différence entre une **entrée** et une **sortie** ? Donnez un exemple de chaque sur la platine.

S2. Qu'est-ce qu'un signal **PWM** ? Quelle grandeur fixe la hauteur du son du buzzer ?

S3. Pourquoi utilise-t-on un **relais** pour commander un ventilateur plutôt qu'une broche du Pico directement ?

Auto-évaluation : cochez votre niveau, le professeur complète

Je sais…	Comp.	Pas encore	En cours	Acquis	Expert	Prof
repérer un module et sa broche sur la platine	**C04**	☐	☐	☐	☐	
installer MicroPython et transférer un programme	**C09**	☐	☐	☐	☐	
commander une sortie : LED, buzzer, relais	**C07**	☐	☐	☐	☐	
lire une entrée : bouton	**C08**	☐	☐	☐	☐	
lire un capteur et afficher une mesure	**C08**	☐	☐	☐	☐	
tester un système et dire s'il est conforme	**C06**	☐	☐	☐	☐	
lire un message d'erreur et corriger mon programme	**C08**	☐	☐	☐	☐	

Avant de ranger la mallette

- ☐ Câbles débranchés
- ☐ Pico laissé sur la platine
- ☐ Tous les éléments dans la mallette (liste p. 2)
- ☐ Poste de travail rangé